MUSEPATH

INTERVIEW TECHNICAL DOSSIER

MUSEPATH: AI-POWERED MUSIC PLANNER

FULL TECHNICAL GUIDE & ARCHITECTURAL OVERVIEW

1. EXECUTIVE SUMMARY & PROJECT SCOPE

MusePath is a specialized, interactive web application designed for self-directed musicians who want customized, AI-generated practice roadmaps. Standard music planning software often relies on generic curriculum paths that fail to account for a user's unique interests, specific practice time, current proficiency level, and musical mood.

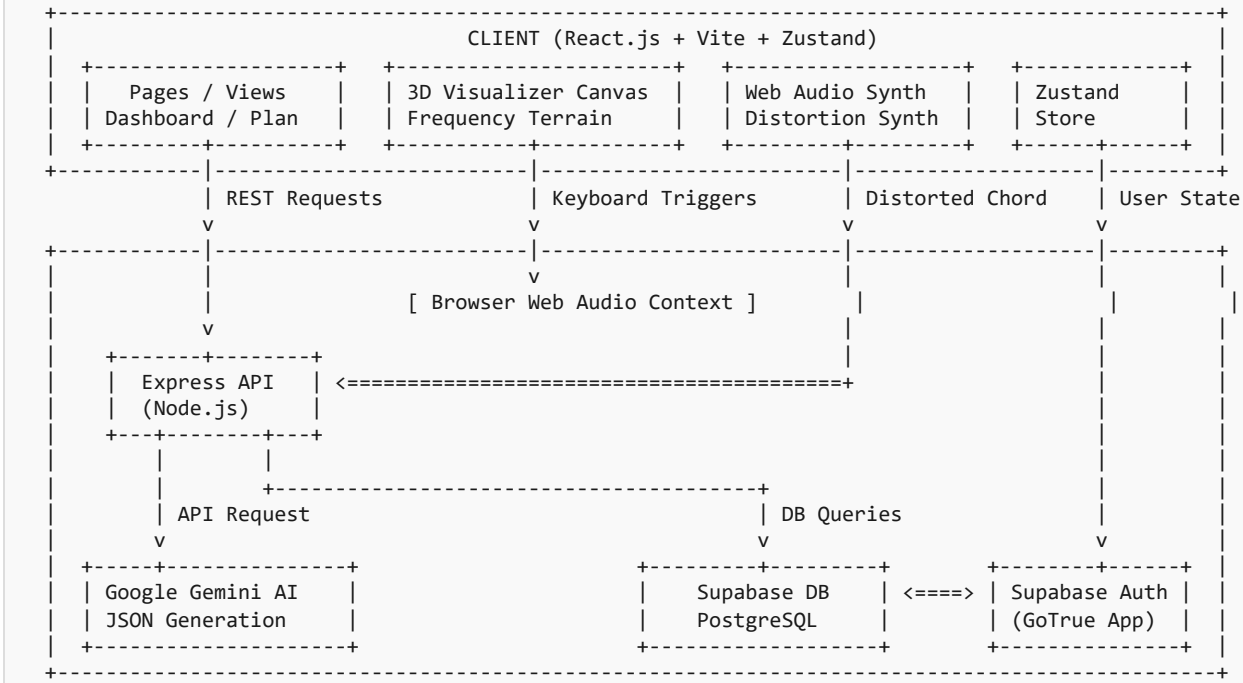
By integrating Google's **Gemini Generative AI** API with a structured database and raw, modern design principles, MusePath generates highly tailored music plans in seconds. Additionally, the application features gamified XP progress tracking, song search and tutorial lookups, and specialized frontend experiences—including a custom **3D frequency terrain visualizer** and a **Web Audio electric guitar distortion synthesizer** to create an engaging, premium user journey.

KEY FEATURES COVERED:

- **AI Planning Engine**: Structured multi-month, multi-week learning roadmaps generated via Gemini Pro using strict JSON structures.
- **Active Plan Switcher**: Support for maintaining, generating, and switching between multiple custom plans.
- **3D Graphics Visualizer**: An interactive, high-performance frequency terrain drawn directly on HTML5 Canvas using projected 3D coordinates.
- **Custom Sound Synth**: Built-in guitar synth utilizing Web Audio API nodes, low-pass speaker cabinet filters, and waveshaper distortion curve algorithms.
- **Full Device Compatibility**: Stateful hamburger navigation, overlays, and typography scaling (using CSS clamp) fully optimized for all viewports.

2. SYSTEM ARCHITECTURE & DATA FLOW

The application follows a decoupled client-server architecture with a relational backend:


CLIENT (React.js + Vite + Zustand)
Pages / Views
Dashboard / Plan
3D Visualizer Canvas
Frequency Terrain
Web Audio Synth
Distortion Synth
Zustand
Store
REST Requests
Keyboard Triggers
Distorted Chord
User State
[Browser Web Audio Context]
Express API
(Node.js)
API Request
DB Queries
Google Gemini AI
JSON Generation
Supabase DB
PostgreSQL
<====>
Supabase Auth
(GoTrue App)

3. DATABASE DESIGN & SUPABASE SCHEMA

The database is hosted on **Supabase (PostgreSQL)**, leveraging core features such as Row Level Security (RLS) and custom triggers to automatically link authenticated users to profile profiles.

DATABASE TABLES ENTITY SUMMARY

TABLE NAME	PRIMARY KEYS / FOREIGN KEYS	KEY FIELDS & TYPES	PURPOSE / ROLE
users	`id` (PK, UUID -> auth.users)	`email`, `streak_days`, `xp`, `instrument`, `skill_level`	Stores profile status, streaks, total practice minutes, and gamification level metrics.
learning_plans	`id` (PK), `user_id` (FK)	`title`, `instrument`, `plan_data` (JSONB), `is_active` (Boolean)	Maintains custom curriculum plans. Multiple plans can exist per user, but only one is `is_active = true`.
learning_weeks	`id` (PK), `plan_id` (FK), `user_id` (FK)	`week_number`, `title`, `topics` (Text[]), `is_completed` (Boolean)	Normalized breakdown of weekly learning milestones, checklist topics, and youtube search metadata.
progress	`id` (PK), `user_id` (FK), `plan_id` (FK)	`completed_weeks`, `total_weeks`, `completion_percentage`	Tracks overall completion percentage of the active roadmap.
practice_logs	`id` (PK), `user_id` (FK), `plan_id` (FK)	`duration_minutes`, `notes`, `practiced_at` (Timestamp)	Maintains history logs of when and how long the user practiced. Displays on the dashboard heatmaps.
saved_songs	`id` (PK), `user_id` (FK)	`song_title`, `song_artist`, `difficulty`, `spotify_id`	Songs that the user saved from the discovery page.
achievements	`id` (PK), `user_id` (FK)	`achievement_key`, `title`, `description`, `earned_at`	Gamified badges awarded dynamically (e.g. creating first plan, logging practice).

4. CORE TECHNICAL IMPLEMENTATIONS

A. SPEED-OPTIMIZED ASYNCHRONOUS API FLOW

Generating multi-month plans requires writing detailed schemas, which took upwards of 15 seconds due to Spotify's metadata search rate limits and network connection limits.

The Optimization: The API was restructured to decouple the plan generation from metadata enrichment. The server invokes Gemini to generate the plan layout, inserts the raw plan details into Supabase, and immediately returns the JSON plan data to the frontend in under 5 seconds. Right after returning, a background task (`setTimeout`) triggers the Spotify API search queries concurrently and updates the database row asynchronously. The frontend UI seamlessly retrieves artwork and preview audio on sub-sequent route fetches without blocking page loads.

B. WEB AUDIO API DISTORTED GUITAR SYNTH

To create the heavy metal audio experience, a Web Audio synthesizer was engineered. The synth stacks multiple detuned sawtooth oscillators to create a thick sound wave, passes the output through a custom Waveshaper distortion node, and shapes the tone using peaking and cabinet low-pass filters:

```
// Stacked Oscillators => Waveshaper Distortion => Cabinet Filter => Destination
const synthChord = (ctx) => {
  const osc1 = ctx.createOscillator();
  const osc2 = ctx.createOscillator();
  const distortion = ctx.createWaveShaper();
  const filter = ctx.createBiquadFilter();

  // Custom Float32 array representing clipping curve
  distortion.curve = makeDistortionCurve(400);
  distortion.oversample = '4x';

  // Lower frequencies to emulate a speaker cabinet response
  filter.type = 'lowpass';
  filter.frequency.value = 2500;

  osc1.type = 'sawtooth'; osc1.frequency.value = 82.41; // Low E
  osc2.type = 'sawtooth'; osc2.frequency.value = 123.47; // B (Fifth)
  osc2.detune.value = 8; // Rich chorus effect

  osc1.connect(distortion);
  osc2.connect(distortion);
  distortion.connect(filter);
  filter.connect(ctx.destination);
};
```

C. 3D FREQUENCY VISUALIZER

Instead of standard libraries, the 3D visualizer renders a stark monochrome terrain on an HTML5 canvas:

- Creates a grid of 3D coordinates `(x, y, z)`.
- Dynamically sets height `z` using frequency bins fetched from an audio analyzer.
- Projects 3D points onto a 2D canvas space using perspective scaling: `x_projected = center_x + (x * perspective_ratio)`.
- Uses coordinate transformation matrices to rotate the mesh grid in response to user mouse movements.

5. HIGH-SCORING INTERVIEW QUESTIONS & ANSWERS

> #### Q1: HOW DOES THE PLAN SWITCHING MECHANISM MAINTAIN STATE INTEGRITY?
>
> **Answer:** The learning plans use a binary `is_active` status column. When a user requests to switch their active roadmap, the database receives a request to set `is_active = false` for all plans belonging to that `userId`, and subsequently sets `is_active = true` for the selected plan. Additionally, the user's base instrument and skill level parameters in the `users` table are automatically synchronized to match the newly activated plan, triggering instant, reactive updates to the dashboard statistics, heatmaps, and video recommendations.

> #### Q2: WHY DID YOU OPT FOR A BACKGROUND SPOTIFY LOOKUP, AND HOW DOES IT PREVENT DATABASE CONFLICTS?
>
> **Answer:** Fetching metadata for up to 96 songs synchronously during plan generation took 15+ seconds, hitting rate limits and causing API timeouts. By moving it to a background execution flow, we return the plan structure generated by Gemini in 4 seconds. In the background, the server asynchronously queries Spotify and saves the updated JSONB data to the database using Supabase's optimistic updates. If a user views the plan before enrichment completes, the app displays a default musical icon fallback, which upgrades to album art as soon as the background task updates the row.

> #### Q3: HOW DID YOU ENSURE TYPOGRAPHY SCALES CLEANLY ON VARIOUS DEVICES?
>
> **Answer:** Standard pixel-based sizing caused large Bebas Neue display titles to wrap line-by-line on mobile screens. I replaced static heading sizing with the CSS `clamp()` function (e.g. `clamp(1.75rem, 5vw, 3.5rem)`). This establishes a responsive range that adapts dynamically based on the viewport width, maintaining a readable scale on small devices while rendering large editorial headlines on desktop monitors.

Q4: HOW DOES THE CUSTOM WEB AUDIO SYNTHESIZER PRODUCE A DISTORTED SOUND?

Answer: The synthesizer uses a `WaveShaperNode` to apply non-linear distortion. We populate a `Float32Array` clipping curve using a mathematical formula: `f(x) = (3 + k)x / (3 + k|x|)`, where `k` is the distortion level. When sound waves pass through, their peaks are clipped, generating high-frequency harmonics (distortion). Stacking detuned sawtooth oscillators adds a thick, chorused tone, and a `BiquadFilterNode` configured as a low-pass filter simulates the cabinet speaker roll-off above 2.5 kHz to keep the high-frequency harmonics from sounding harsh.